

1 Arrays

Arrays are added at the expression level by means of the following extension:

$$\begin{aligned} \mathcal{E} \quad + = \quad & [\mathcal{E}^*] \\ & \mathcal{E}[\mathcal{E}] \\ & \mathcal{E} . \text{length} \\ & \text{elemRef } \mathcal{E}[\mathcal{E}] \end{aligned}$$

Additional well-formedness rules for the new constructs:

$$\begin{array}{c} \frac{e_1 : \mathbf{Val} \dots e_k : \mathbf{Val}}{[e_1, \dots, e_k] : \mathbf{Val}} \quad \frac{e_1 : \mathbf{Val} \dots e_k : \mathbf{Val}}{[e_1, \dots, e_k] : \mathbf{Weak}} \quad \frac{e_1 : \mathbf{Val} \dots e_k : \mathbf{Val}}{\mathbf{ignore} [e_1, \dots, e_k] : \mathbf{Void}} \\[10pt] \frac{e : \mathbf{Val} \quad i : \mathbf{Val}}{e[i] : \mathbf{Val}} \quad \frac{e : \mathbf{Val} \quad i : \mathbf{Val}}{e[i] : \mathbf{Weak}} \quad \frac{e : \mathbf{Val} \quad i : \mathbf{Val}}{\mathbf{ignore} e[i] : \mathbf{Void}} \\[10pt] \frac{e : \mathbf{Val} \quad i : \mathbf{Val}}{\mathbf{elemRef} e[i] : \mathbf{Ref}} \end{array}$$

2 Operational Semantics

An array can be represented as a pair: the length of the array and a mapping from indices to elements. If we denote X the set of elements then the set of all arrays $\mathcal{A}(X)$ can be defined as follows:

$$\mathcal{A}(X) = \mathbb{N} \times (\mathbb{N} \rightarrow X)$$

For an array $\langle n, f \rangle$ we assume $\text{dom } f = [0..n-1]$. An element selection function:

$$\begin{aligned} \bullet[\bullet] : \mathcal{A}(X) &\rightarrow \mathbb{N} \rightarrow X \\ \langle n, f \rangle [i] &= \begin{cases} f(i) & , \quad i < n \\ \perp & , \quad \text{otherwise} \end{cases} \end{aligned}$$

We represent arrays by *references*. Thus, we introduce a (linearly) ordered set of locations

$$\mathcal{L} = \{l_0, l_1, \dots\}$$

Now, the set of all values the programs operate on can be described as follows:

$$\mathcal{V} = \mathbb{Z} \mid \mathcal{L}$$

To access arrays, we introduce an abstraction of memory:

$$\mathcal{M} = \mathcal{L} \rightarrow \mathcal{A}(\mathcal{V})$$

We now add two more components to the configurations: a memory function μ and the first free memory location l_m , and define the following primitive

$$\mathbf{mem} \langle \sigma, \omega, \mu, l_m \rangle = \mu$$

which gives a memory function from a configuration.

The definition of state does not change, hence all existing rules are preserved (modulo adding additional components to configurations) The rules for the new kinds of expressions are as follows:

$$\begin{array}{c}
\frac{c \xRightarrow{e_0, \dots, e_k} \langle \langle \sigma, \omega, \mu, l \rangle, v_1, \dots, v_k \rangle}{c \xRightarrow{[e_0, \dots, e_k]} \langle \langle \sigma, \omega, \mu[l \leftarrow \langle k+1, i \mapsto v_i \rangle], l+1 \rangle, l \rangle} \quad [\text{ARRAY}] \\
\\
\frac{c \xRightarrow{ei} \langle c', lv \rangle \quad l \in \mathcal{L} \quad v \in \mathbb{Z}}{c \xRightarrow{e[i]} \langle c', ((\mathbf{mem} \ c')(l))[i] \rangle} \quad [\text{ARRAYELEM}] \\
\\
\frac{c \xRightarrow{ei} \langle c', lv \rangle \quad l \in \mathcal{L} \quad v \in \mathbb{Z}}{c \xRightarrow{\mathbf{elemRef} \ e[i]} \langle c', \mathbf{elemRef} \ l \ v \rangle} \quad [\text{ARRAYELEMREF}] \\
\\
\frac{c \xRightarrow{e} \langle c', l \rangle \quad l \in \mathcal{L}}{c \xRightarrow{e.\mathbf{length}} \langle c', \mathbf{fst}(\mathbf{mem} \ c')(l) \rangle} \quad [\text{ARRAYLENGTH}]
\end{array}$$

We also need one additional rule for assignment:

$$\frac{c \xRightarrow{lr} \langle \langle \sigma, \omega, \mu, l \rangle, [\mathbf{elemRef} \ a \ i]v \rangle \quad a \in \mathcal{L}}{c \xRightarrow{l := r} \langle \langle \sigma, \omega, \mu[a \leftarrow \langle \mathbf{fst} \ \mu(a), (\mathbf{snd} \ \mu(a))[i \leftarrow v] \rangle], l \rangle, v \rangle} \quad [\text{ASSIGNARRAY}]$$

3 Stack Machine

In stack machine we add the following new instructions:

$$\begin{array}{lcl}
\mathcal{I} & + = & \text{ARRAY } \mathbb{N} \\
& & \text{ELEM} \\
& & \text{STA}
\end{array}$$

We also add memory function and current location components to the configuration; as state components are preserved, all rules are preserved as well. The new rules are:

$$\frac{\langle s[n, \dots], s_c, \sigma, \omega, \mu[l \leftarrow \langle n, i \mapsto s[n-i-1] \rangle], l+1 \rangle \xRightarrow{P} c : P}{\langle s, s_c, \sigma, \omega, \mu, l \rangle \xRightarrow{[\text{ARRAY } n]P} c : P} \quad [\text{ARRAY}_{\mathcal{S.M.}}]$$

$$\begin{array}{c}
\frac{\langle [(\mu(a))[i]]s, s_c, \sigma, \omega, \mu, l \rangle \xRightarrow[\mathcal{SM}]{P} c : P}{\langle ias, s_c, \sigma, \omega, \mu, l \rangle \xRightarrow[\mathcal{SM}]{[\text{ELEM}]P} c : P} \quad [\text{ELEM}_{\mathcal{SM}}] \\
\\
\frac{\langle vs, s_c, \sigma, \omega, \mu[a \leftarrow \langle \mathbf{fst}(\mu(a)), (\mathbf{snd}(\mu(a)))[i \leftarrow v] \rangle], l \rangle \xRightarrow[\mathcal{SM}]{P} c : P}{\langle vias, s_c, \sigma, \omega, \mu, l \rangle \xRightarrow[\mathcal{SM}]{[\text{STA}]P} c : P} \quad [\text{STA}_{\mathcal{SM}}]
\end{array}$$